HYDRA — Technical Audit, Completion Assessment & Commercial Readiness Report

Independent read-only audit of **MoallaMelek/Hydra-Agent-Orchestrator**

Audited commit	`38bc58c8991540940af5f32ed7eca5194fcce2f1` (2026-10-10, `main`, sole branch, 8 commits, clean tree)
Package version	hydra 0.2.61
Upstream origin	`github.com/jpdlr/hydra` (public, MIT) — fork point upstream commit `d8ad561` (2026-05-11)
Audit date	2026-10-12
Auditor	Claude Opus 5.5 (read-only). Independent challenge pass: a fresh, context-less Codex CLI 0.154.0 session in a read-only sandbox; its corrections are incorporated and attributed.
Environment	Windows 11 Pro 10.0.26200 · Node 22.23.2 · npm 10.9.8 · git 2.51.2 · Claude Code 2.1.282 · Codex CLI 0.154.0 (native `codex.exe` from the Codex app bin directory) · Edge (PDF rendering)
Method	Repository clone to an isolated path; static review; typecheck/lint/tests/builds; dependency audit; deterministic module probes; live end-to-end runs of Hydra's real TaskCoordinator against throwaway git fixtures in the temp directory; inspection of the deployed PWA bundle and GitHub metadata. No changes were made to Hydra, no commits or pushes, nothing was deployed, and no Firebase data was touched.
Evidence tags	**VERIFIED** observed by execution or exact code path · **INFERRED** derived from code, not executed · **BLOCKED** could not test · **UNKNOWN**

Companion file: `HYDRA_AUDIT_BACKLOG.md` (machine-readable prioritized backlog with acceptance criteria). Nothing in this report is legal advice.

1. Executive summary

> **Bottom line.** Hydra contains one genuinely working, well-designed core: a **coordinated task pipeline** that, in this audit, autonomously fixed a real bug end to end. Codex wrote the fix, Claude reviewed it independently, Codex ran the project's tests, and Hydra certified completion from *observed exit codes*, not model claims. Around that core, **Hydra is not a releasable product**. About 90% of the TypeScript is inherited from the public MIT project `jpdlr/hydra` without attribution in the README. Remote Control cannot work in released builds and the live mobile app runs on the upstream maintainer's Firebase project. CI is red, no installable release exists, verification only supports npm projects, and production dependencies carry critical advisories. **Recommendation: do not sell Hydra Core today. Do not list anything on Agensi until P0 items are fixed; the fastest honest commercial path is 2–3 small SKILL.md skills extracted from Hydra's best ideas.**

What is real

- VERIFIED Live run: Claude+Codex "fix the bug" task reached `completed` in 158 s; an independent re-run of the tests passed; the diff was one line.
- VERIFIED Safe-mode approval gate stops before any write; denying leaves the tree untouched; cancellation stops providers in about 1 s.
- VERIFIED 236/236 unit tests pass (44 files); typecheck and production build pass; PWA and Cloud Functions compile.
- VERIFIED Solid security baseline in the desktop shell: context isolation and sandboxed renderer, bearer-token auth on the daemon and MCP, zod-validated requests, symlink-aware path containment, no shell for headless CLIs, YOLO/full-access hard-disabled.

What is not

- VERIFIED Provenance: the fork added about 3.5k TS lines to about 32.1k upstream lines; the README presents the whole as original.
- VERIFIED Remote Control: packaged builds never embed the Firebase config (sentinel build); the deployed PWA uses `hydra-za` (the upstream author's project) and an outdated protocol.
- VERIFIED No GitHub releases, the Homebrew tap returns 404, CI fails on lint for the last 3 pushes, and builds are unsigned.
- VERIFIED Verification plans exist only for npm scripts. Python, Rust and Go yield an empty plan. A Claude-only implementation run ended `blocked` even though its fix was correct.
- VERIFIED DEMO stage and final APPROVAL (accept/reject/rollback) do not exist. Tasks write directly into the user's working tree.
- VERIFIED `npm audit` (production): root 38 (3 critical/16 high), PWA 14 (2 critical), Functions 23 (3 critical). Electron 33 is past its support window.

Stage: functional prototype (core pipeline) inside an inherited beta-quality desktop app. **Estimated completion toward a commercially distributable Hydra Core: ~35%** (engineering estimate, not a measurement). **Remaining effort:** ~165 / 330 / 570 h (optimistic / realistic / pessimistic) for Hydra Core; ~35 / 60 / 105 h for a first set of Agensi skills.

2. Project overview

Hydra is an Electron 33 + React 18 desktop app that runs Claude Code and Codex CLI agents. It has three execution models and one remote channel:

Subsystem	What it is	Origin
Interactive agents	One node-pty PTY per agent running the real `claude` / `codex` TUI; chat and grid views; session import and resume	Upstream
Manager MCP	Loopback HTTP MCP server with 7 tools (list/create/prompt/output/broadcast/kill/restart) used by a "manager" Claude agent	Upstream; bearer auth added in fork
Headless runs	Non-interactive `claude -p --output-format stream-json` / `codex exec --json` runs with JSONL logs	Upstream; confinement flags added in fork
Coordinated tasks	Durable multi-provider pipeline: analysis → (approval) → implementation → review → repair → verification → synthesis	**Fork-original** (~1,219 lines incl. tests)
Remote Control	Firebase (Auth custom tokens, Firestore inbox/outbox/state, callable Functions) + "Hydra Remote" PWA via QR code	Upstream; rules/roles hardened in fork
Detached daemon	Electron-as-Node process over a named pipe/Unix socket; renderer → preload IPC → main → DaemonClient → daemon	Upstream; schemas and token rotation added in fork

Provenance (VERIFIED)

Measure	Value
Upstream	`jpdlr/hydra`, public, MIT, created 2026-02-07, 268 commits; `main` unchanged since 2026-05-11; a side branch was pushed 2026-10-12
Fork point	`d8ad561` — named explicitly in `docs/debug/HYDRA-DEEP-AUDIT.md:3` ("then a clone of the upstream project")
TS/TSX lines at fork point	32,105
Fork changes (TS/TSX)	+3,517 / −695 lines; overall 113 files, +4,439 / −804 (excluding lockfiles and images)
Fork-added modules	`electron/tasks/*`, `electron/security/*`, `cliExecution.ts`, `requestSchemas.ts`, `shared/tasks.ts`, `src/components/Tasks/*`, PWA `TaskList.tsx`, 15 new test files
Who wrote the fork changes	Per `.ai/handoffs/final.md` and `.ai/current-task.md`: Codex CLI implemented, with Codex/Claude review; directed by the owner
Licence	MIT, with both copyright lines retained (`LICENSE:3-4`). Resale is legally permitted under MIT; the issue is attribution and credibility.

3. Verified architecture

Process model. Electron main (`electron/main.ts`) creates a sandboxed renderer (`main.ts:68-73`: `contextIsolation`, `sandbox`, `webSecurity` on, `nodeIntegration` off) and connects to a detached daemon (`electron/daemon/index.ts`). The daemon authenticates every HTTP and WebSocket request with a 32-byte bearer token compared in constant time (`DaemonServer.ts:107-114, 171-172, 455`). The token is stored in a lock file (`lock.ts`; the 0600 mode has no effect on Windows). The daemon owns AgentManager, HeadlessOrchestrator, TaskCoordinator, session catalogs and the MCP server. RemoteControlService lives in Electron *main*, not the daemon, so Remote Control ends when the window process exits.

Coordinated task flow (`electron/tasks/TaskCoordinator.ts`): `create` → `launch` (at most 4 concurrent, 40/60 min total budget) → `execute`: provider availability → project snapshot (`projectContext.ts`, ≤240 files, 90 KB, credential paths excluded) → parallel independent analyses (`:173`) → intent routing (`:180-188`) → writer selection by self-reported confidence (`:190-192`) → Safe-mode approval (`:207-210`) → per-project writer lease (`:26-52, 212`) → implementation (`:220`) → review loop with MAX_REPAIRS=1 (`:225-247`) → verification with evidence validation (`:253-262`) → fingerprint re-check (`:263-264`) → synthesis (`:265-276`). Provider runs go through `providerAdapters.ts` → `HeadlessOrchestrator.start` → `spawnResolvedCli` (no shell; `cliExecution.ts:36-41`). Completion is detected from provider JSON events (`semanticResults.ts`), never from exit code alone.

Engineering quality

Dimension	Assessment
Separation of concerns	Good at the process level (renderer/main/daemon). Weak inside modules: `AgentManager.ts` (1,134 lines) and `ipc/handlers.ts` (1,133) are monoliths. Four overlapping orchestration paths (interactive, Manager MCP, headless, tasks) with different safety models.
Readability	TaskCoordinator logic is compressed into very long single-line statements (e.g. `:173, :229, :256, :274`). Hard to review and extend; prompts are inlined string literals.
Error handling	Generally explicit in the task path (typed failures, blocked states with reasons). Swallowed errors in remote delivery (`DaemonClient.ts:244-246`) and in many best-effort persistence paths.
State and persistence	Atomic JSON writes; active tasks become `interrupted` after a daemon restart (`:73-77`), with no automatic resume. The whole store is rewritten on every save (`:334-338`).
Process lifecycle	Process-tree termination via `taskkill /T /F` and POSIX process groups (`cliExecution.ts:44-60`). Cancellation verified live.
Concurrency	One writer per canonical project root; interactive writers are blocked while a task owns the project (`AgentManager.ts:465-469`). Analyses run in parallel.
Observability	JSONL logs per headless run; task events (last 500); structured app log. No metrics or cost accounting.
Cross-platform	Windows-specific executable resolution (`cliExecution.ts:7-34`); live validation exists only on Windows.

Genuine orchestrator or a collection of scripts? The coordinated-task subsystem is a **genuine, if small, orchestrator**: a durable state machine with role separation, independent analyses, a writer lease, bounded repair, evidence-gated verification and integrity checks. The rest of Hydra is a **multi-terminal manager**: it spawns CLIs and pipes text into PTYs, with heuristic status detection. Both are fair descriptions of different halves of the codebase.

4. Feature inventory

Category	Features
1. Exists and works (VERIFIED)	Coordinated analysis with Claude+Codex · Safe-mode pre-write approval gate · coordinated implementation (Codex writer, Claude reviewer, npm verification) · cancellation · CLI detection for Claude 2.1.282 and Codex 0.154.0 · typecheck/tests/production build · PWA and Functions compile · outbox payload byte-bounding (806 KB → 48 KB) · daemon/MCP token auth (code + tests)
2. Implemented, not verified in this audit	Interactive PTY agents (chat/grid views, broadcast) · session import and resume · Manager MCP orchestration (the repo's own state doc says "NOT VERIFIED") · headless panel · Git panel and PR review (`gh`) · editor and file tree · usage dashboard · notifications · workspace persistence · daemon crash recovery · free terminals · skills enable/disable tab · desktop task UI (component tests only; GUI not launched)
3. Partially implemented	Remote Control (prompt-only, read-only task list, packaged-config defect, third-party backend) · verification (npm only) · approval (pre-write, Safe mode only) · Claude-only implementation (blocked in live run) · Firebase security (rules hardened in repo but untested and not deployed by the owner) · cost controls (time and attempt limits only)
4. Documented but absent or broken	YOLO mode (README:59-62; disabled at `AgentManager.ts:916-921`) · GitHub Releases / Homebrew / winget / AUR installs · Windows in-app updates (no releases to update from) · phone stop/restart (deployed PWA sends them; desktop drops them) · "signed" release builds (signing disabled)
5. Entirely missing	DEMO stage · final APPROVAL (accept/reject) · rollback / worktree isolation for tasks · remote approvals · push notifications · persistent device pairing · per-task token/cost budgets · non-npm verification · any SKILL.md skill · macOS/Linux validation · UI E2E tests · Firebase emulator tests

5. Agent orchestration analysis

The requested lifecycle is PLAN → IMPLEMENT → REVIEW → FIX → VERIFY → DEMO → APPROVAL. Hydra's actual phases are `planning, awaiting_approval, queued, implementation, review, repair, verification, synthesis, completed|blocked|failed|canceled|interrupted` (`shared/tasks.ts:4`).

Stage	Implemented	Agent / invocation	Input → output	Completion / failure	Human
PLAN	**Yes**	Every selected provider, in parallel, read-only (`:173`)	Task context JSON + snapshot → JSON {intent, plan, writerConfidence, decisions}	Semantic result event; all failed → task fails	No
APPROVAL (pre-write)	**Safe mode only**	—	Approval record {reason, scope=root}	Approve → queued; deny → blocked (**verified live**)	Yes (desktop only)
IMPLEMENT	**Yes**	Writer = highest-confidence eligible provider. Codex `exec -s workspace-write`, approval never, network off; Claude `--restricted --safe-mode --tools Read,Glob,Grep,Write,Edit --permission-mode acceptEdits`	Context + snapshot → edits in the working tree	Semantic completion required; 10/20 min run timeout	No
REVIEW	**Yes**	A different provider when available, otherwise the same provider (labelled)	Fresh snapshot → JSON {verdict, findings}	Invalid JSON → blocked; unresolved after repair budget → blocked	No
FIX	**Yes, 1 round**	Writer (`:246`)	Findings → edits	MAX_REPAIRS=1 then re-review	No
VERIFY	**npm only**	Codex if selected, otherwise Claude Read-only inspection	Plan from package.json → observed command evidence	Every planned command must show exit 0 in tool events; fingerprint must match reviewed state	No
DEMO	**No**	No code, phase or UI exists.			
APPROVAL (final)	**No**	No accept/reject, diff gate or rollback. Results land directly in the working tree; Developer and Autonomous modes skip all human gates.			

Cross-cutting checks

Concern	Finding
Role separation and review independence	Good: a different provider is preferred for review and synthesis, and same-provider review is labelled. Writer choice relies on self-reported confidence.
Agent-to-agent communication and context sharing	Indirect, via a JSON task context with the last 10 attempts, findings and decisions (`:280-289`). No direct agent messaging. Agents are told not to spawn other agents.
Retries and infinite-loop prevention	Bounded: 1 repair, 12 attempts per turn, 200 messages per task, 4 concurrent tasks, time budgets. Retry is manual.
Timeout and cancellation	Verified live (about 1 s). Timeout shows as "Stopped", and cancel-during-analysis shows a misleading "All independent analyses failed".
Crash recovery	Active tasks become `interrupted` on restart, with no replay (by design). Not executed in this audit.
Token and cost controls	**None** beyond time and attempt limits.

Prompt injection exposure	Repository content (including AGENTS.md and README) is embedded in every prompt. Mitigations: Codex OS sandbox with network off; Claude tool allowlist; verification evidence from tool events. Residual risk: injected instructions can steer edits inside the project.
Unsafe command execution	Codex runs arbitrary project scripts (`npm run test`) inside its sandbox, including for untrusted repositories. Verification runs with write access (`providerAdapters.ts:27`).
Human approval enforcement	Enforced server-side in the coordinator (`:116-121, 207-210`) for Safe mode; approval never carries over to new turns (`:127-128`).

Can Hydra complete a real development task autonomously? VERIFIED Yes, for a small, well-specified change in an npm project with both CLIs installed and authenticated (live run below). It cannot certify completion for non-npm projects, and Claude-only implementation was unreliable in testing. Larger multi-file tasks were not tested.

6. Frontend, mobile and remote control

Desktop

BLOCKED The Electron GUI was not launched in this audit. Evidence comes from code and passing component tests (`TaskWorkspace.test.tsx`, `App.flow.test.tsx` and others). The task workspace offers project selection, permission mode (Safe/Developer/Autonomous), intent, attachments (credential-like files rejected, `TaskWorkspace.tsx:88`), an approval card and stop. Stale YOLO UI remains (`App.tsx:962`).

Mobile PWA and remote control

Check	Result
Authentication	Custom tokens with `sessionId` + `role` claims, checked client-side (`useRemoteSession.ts:192-196`) and by rules (`firestore.rules:8-27`). INFERRED correct in repo; not evaluated by an emulator.
Session persistence and reconnection	Pairing link stored in localStorage; auto-reconnect while the same desktop session lives; heartbeat staleness 45 s. A new desktop launch requires a new QR code.
Remote task submission	**No.** The phone can only send a raw prompt to an interactive agent. Tasks are visible read-only.
Real-time updates	Raw PTY output flushed every 2 s to the outbox; heuristic "streaming" indicators.
Approve/reject from phone	**No.** Remote approvals are deliberately local-only for tasks. But raw text plus two Enters into a PTY can confirm CLI permission dialogs (INFERRED).
Error feedback	Rejected prompts are reported as delivered (`DaemonClient.ts:244-246`) — VERIFIED defect.
Released-build operation	**Fails** — Firebase config is not embedded (sentinel build) and `HYDRA_REMOTE_APP_URL` is never set.
Deployed instance	Live at `hydra-remote-moalla-260912.web.app` (HTTP 200) but authenticates against `hydra-za` (upstream author's project) and uses the old protocol (`get_history/stop/restart`), which the current desktop silently drops.
Unauthorized execution	A stolen pairing link gives prompt injection into non-YOLO agents for up to 24 h. YOLO agents are excluded (`RemoteControlService.ts:456, 470`).

Verdict: mobile control is a **demonstration that worked on the developer's machine against someone else's backend**. It is not a functional feature for a real user today. Blockers: own backend (P0-02), config embedding (P0-03), protocol sync (P1-10), delivery acknowledgement (P1-06) and PTY approval safety (P1-07).

7. Security audit

Question: could installing Hydra expose a developer's machine, repositories, credentials or files? **Local-only use has a thoughtful baseline. It is not yet safe to hand to strangers**, because of unpatched critical dependencies, an end-of-life Electron, unsigned binaries, direct writes to repositories, and a remote path wired to a third-party backend.

Sev.	Finding	Evidence	Tag
Critical	Live Remote Control sends prompts and raw terminal output (which can contain secrets) to a Firestore database owned by a third party (`hydra-za`)	Deployed bundle authDomain; upstream commit `15d6041`	VERIFIED
High	Critical/high advisories in production deps (protobufjs ACE, websocket-driver, proxy-addr, ws, MCP SDK); Electron 33 past support	`npm audit --omit=dev`; `package.json:43-61`	VERIFIED (exploitability in context INFERRED)
High	Remote raw input can confirm interactive permission dialogs	`AgentManager.ts:984-998`; `RemoteControlService.ts:454-459`	INFERRED; Codex: plausible
High	Unauthenticated `createSession` lets anyone mint sessions and tokens (billing abuse; no access to existing sessions)	`functions/src/index.ts:19-70`	VERIFIED code
High	Developer/Autonomous tasks write directly to the user's working tree with no final gate or rollback	`TaskCoordinator.ts:207-220`	VERIFIED
High	Unsigned binaries; macOS hardened runtime off	`electron-builder.yml:20-30`	VERIFIED
Medium	`.env` in the launch directory is merged into the environment of every child agent	`load-env.ts:38-47`; `AgentManager.ts:478`	VERIFIED path / INFERRED exploit
Medium	Runtime `npx -y @ccusage/codex@latest` (unpinned remote code)	`handlers.ts:665-677`	VERIFIED
Medium	Verification step has write access; integrity fingerprint is partial	`providerAdapters.ts:27`; `projectContext.ts:23-61`	VERIFIED (Codex finding)
Medium	Any directory (home, drive root) accepted as a project root	`pathPolicy.ts:15-20`; `McpServer.ts:272-290`	VERIFIED
Medium	Prompt injection from repository content into agent prompts	`projectContext.ts`; `TaskCoordinator.ts:173,220`	INFERRED; mitigated by sandboxing
Medium	Pairing credential persisted in PWA localStorage (valid up to 24 h)	`useRemoteSession.ts:81-113`	VERIFIED
Low	Git branch names are not separated with `--` (renderer-controlled); lock-file 0600 mode ineffective on Windows; spawn args logged to the console	`GitService.ts:254-266`; `lock.ts:41`; `AgentManager.ts:497`	VERIFIED

Positive controls (VERIFIED in code)

- Renderer hardening and navigation blocking (`main.ts:68-73`, `security/navigation.ts`).
- Constant-time bearer-token auth on daemon and MCP; MCP bound to 127.0.0.1. The September audit's "unauthenticated MCP" finding is fixed.
- Zod schemas on daemon routes; symlink-aware containment for file reads and writes.
- Headless CLIs spawned without a shell; prompts sent on stdin.
- YOLO/full-access disabled; Codex `approval_policy="never"` with network off; Claude `--restricted --safe-mode --strict-mcp-config` with a tool allowlist.
- Credential paths excluded from context; secret-like attachments rejected.

- Firestore rules with role separation, expiry and inbox schema in the repo (deployment unknown).
- Recursive session delete via a host-only callable.

8. Testing results

#	Test	Expected	Actual	Result
T01	Clone and history	Readable history	8 commits, 1 branch, clean; first commit already v0.2.61	**PASS**
T02	`npm ci` (root)	Installs	983 packages, 3 min	**PASS**
T03	`npm run typecheck`	Exit 0	Exit 0 (44 s)	**PASS**
T04	`npm run lint`	Exit 0	6 errors, 2 warnings (same as CI)	**FAIL**
T05	`vitest run`	All pass	236/236 tests, 44 files, 6m45s	**PASS**
T06	`npm run build`	Exit 0	Exit 0 (2m09s); renderer chunk 10.35 MB	**PASS**
T07	PWA `npm ci && npm run build`	Exit 0	Exit 0	**PASS**
T08	Functions `npm ci && npm run build`	Exit 0	Exit 0	**PASS**
T09	`npm audit --omit=dev` ×3	0 critical/high	Root 38 (3C/16H) · PWA 14 (2C/2H) · Functions 23 (3C/7H); root all deps 74 (6C/41H)	**FAIL**
T10	GitHub CI on main	Green	Last 3 runs fail at Lint (runs 37461389670, 37837336393, 38086741427)	**FAIL**
T11	Releases and Homebrew tap	Installable artefacts	0 releases; `MoallaMelek/homebrew-hydra` 404	**FAIL**
T12	Sentinel build: Firebase env embedded	Sentinel present in `out/`	0 occurrences; runtime `process.env.MAIN_VITE_*` lookups remain	**FAIL**
T13	Claude CLI detection	`--tools --restricted --safe-mode` present	All present in 2.1.282; native `claude.exe` resolved	**PASS**
T14	Codex CLI detection	`exec`, `-s`, `--json`, `--disable`	Present in 0.154.0; native `codex.exe` resolved	**PASS**
T15	LIVE analyze (Claude+Codex, Safe)	Completed, no writes	Completed in 54 s; both analyses plus synthesis; tree clean	**PASS**
T16	LIVE Safe-mode gate (implement)	Pause before writes; deny → blocked	`awaiting_approval` after 19 s; tree clean; deny → `blocked`	**PASS**
T17	LIVE implement (Claude+Codex, Developer)	Fix bug, review, verify, complete	Completed in 158 s; writer Codex; reviewer Claude passed; Codex ran `npm run test` exit 0; independent `node --test` exit 0; diff 1 line	**PASS**
T18	LIVE implement (Claude only)	Complete with limited verification	Fix was correct (independent test exit 0) but review flagged "tests not run" twice → `blocked` after 1 repair (119 s)	**FAIL**
T19	LIVE cancellation	Canceled quickly, no writes	Canceled about 1.06 s after request; tree clean; misleading error text	**PASS**
T20	Verification plan for Python/Rust/Go	Plan exists	Empty plan for all three; `npx` scripts dropped	**FAIL**
T21	Watch-mode test script	Rejected or guarded	`"test":"vitest"` accepted (hang risk)	**FAIL (risk)**
T22	Outbox byte bound	≤50 KB	806,718 B → 48,453 B, 12 messages kept	**PASS**
T23	Deployed PWA ownership and protocol	Owner's backend, current protocol	Uses `hydra-za`; old protocol; no task list	**FAIL**

T24	Firestore rules under emulator	Rules enforce roles and expiry	No emulator suite; repo tests string-match only	**BLOCKED**
T25	Firebase connectivity and deployed rules	—	No owner credentials; production data not touched by policy	**BLOCKED**
T26	Phone → desktop end-to-end	Prompt delivered	Requires own backend; packaged config defect	**BLOCKED**
T27	Desktop GUI launch and UX	—	Not launched in this audit	**NOT TESTED**
T28	Interactive PTY agents, Manager MCP, crash recovery, macOS/Linux	—	Unit tests only	**NOT TESTED**

Reproduction. Audit clone at `C:\hydra-audit\src`. Logs in `C:\hydra-audit\logs\`. Live harness: `node C:\hydra-audit\audit-scripts\live-e2e.mjs C:\hydra-audit\src out.json analyze-both safe-gate implement-both-dev implement-claude-only cancel`. The harness creates git fixtures in `%TEMP%` (an `add()` that subtracts, plus a failing `node --test`) and drives Hydra's real TaskCoordinator, HeadlessOrchestrator and provider adapters. Probes: `node C:\hydra-audit\audit-scripts\probes.mjs C:\hydra-audit\src`. Sentinel: build with `MAIN_VITE_HYDRA_FIREBASE_API_KEY=AUDITSENTINEL_APIKEY`, then `grep -r AUDITSENTINEL out/`. Live runs consumed the owner's Claude and Codex subscription usage.

9. Completion scores

Dimension	Score	Justification
Core functionality	**6/10**	Real autonomous fix verified end to end (T17); analysis, gate and cancel verified. npm-only verification (T20); Claude-only path failed (T18); large tasks untested.
Agent orchestration	**6/10**	Durable state machine, evidence-gated verification, writer leases, bounded loops. No DEMO or final APPROVAL, one repair round, no cost control, verifier has write access.
Architecture	**6/10**	Clean process separation and validated boundaries. Monolith files, dense one-line logic, four overlapping orchestration paths, whole-store rewrites.
Reliability	**4/10**	Tests green locally but CI red; swallowed remote errors; Windows-only validation; interrupted tasks not resumable; persistence scaling.
Security	**4/10**	Good desktop and daemon controls, offset by critical dependency advisories, EOL Electron, third-party backend, unsigned builds, `.env`/npx issues.
Frontend and UX	**4/10**	Low confidence (GUI not launched). Component tests pass; stale YOLO UI; core workflow undocumented.
Remote control	**2/10**	Cannot work in released builds; live instance on someone else's project with an outdated protocol; prompt-only.
Testing	**5/10**	236 meaningful unit tests (including real CLI-event parsing). No E2E in CI, Firebase tests are string matches, CI not enforced.
Documentation	**3/10**	README misstates YOLO, installs and mobile features, and omits the main new feature. Internal `.ai/` and `docs/debug` notes are candid and useful.
Deployment readiness	**2/10**	No releases, missing tap, unsigned, release config defect, CI red.
Commercial readiness	**1/10**	Unattributed fork of a free public MIT project, no SKILL.md assets, dependency on a third-party backend, nothing installable.

Overall completion: ~35% toward a commercially distributable Hydra Core. *Engineering estimate, not an objective measurement.* The local coordinated pipeline alone is about 60% of a credible MVP.

10. Missing functionality and defects

Confirmed defects: packaged Remote config (P0-03) · third-party backend in production (P0-02) · CI lint failure (P0-04) · no installable release (P0-05) · README inaccuracies (P0-07) · lost remote prompts (P1-06) · Claude-only blocked in review (P1-02, 1/1 runs) · Firestore write amplification (P1-08) · PWA/desktop protocol mismatch (P1-10) · enable-timeout race (P1-16) · misleading cancel text (P2-04).

Suspected risks: dependency advisories (P0-06) · unauthenticated createSession (P0-08) · cwd `.env` injection (P0-09) · unpinned npx (P0-10) · verifier write access (P1-04) · partial fingerprint (P1-05) · PTY permission confirmation via remote (P1-07) · persistence scaling (P1-13) · watch-mode hang (P1-14) · unrestricted project root (P1-15).

Missing features: non-npm verification (P1-01) · final approval, worktree isolation and rollback (P1-03) · cost budgets (P1-11) · signing (P1-12) · cross-platform validation (P1-17) · DEMO (P2-01) · pairing and push (P2-05) · UI E2E (P2-08) · Agensi skills (P3-01).

11. Prioritized engineering backlog

Full details (description, files and lines, dependencies, solution, effort, acceptance, verification) are in `HYDRA_AUDIT_BACKLOG.md`. Effort is shown as optimistic / realistic / pessimistic hours.

ID	Title	Sev.	Effort h
P0-01	Provenance and attribution of forked code	**Critical**	2/4/8
P0-02	Own Firebase backend; retire third-party dependency	**Critical**	6/12/20
P0-03	Embed Firebase and remote URL config in release builds	**Critical**	1/3/5
P0-04	Fix lint and enforce green CI	**High**	0.5/1/2
P0-05	Ship an installable release; fix install docs	**High**	4/10/20
P0-06	Upgrade vulnerable and EOL dependencies (Electron, firebase, ws, MCP SDK)	**High**	8/16/32
P0-07	Make README truthful; remove YOLO remnants	**High**	3/6/10
P0-08	App Check and rate limiting on createSession	**High**	4/8/16
P0-09	Stop cwd `.env` propagation to agents	**Medium**	1/2/3
P0-10	Remove unpinned `npx @latest`	**Medium**	1/2/4
P1-01	Multi-ecosystem verification	**High**	12/24/40
P1-02	Fix Claude-only review blocking	**High**	2/4/8
P1-03	Worktree isolation + final accept/discard (APPROVAL)	**High**	16/28/45
P1-04	Read-only verification sandbox	**Medium**	4/8/14
P1-05	Full-tree integrity fingerprint	**Medium**	3/6/10
P1-06	End-to-end remote delivery acknowledgement	**Medium**	2/4/6
P1-07	Block remote input at permission prompts	**High**	8/16/28
P1-08	Change-only Firestore task sync	**Medium**	1/3/5
P1-09	Firestore rules emulator tests	**Medium**	4/8/12
P1-10	Versioned remote protocol; redeploy PWA	**Medium**	2/4/8
P1-11	Per-task token and cost budgets	**Medium**	6/12/20
P1-12	Code signing and notarization	**Medium**	6/12/24
P1-13	Scalable task persistence	**Medium**	6/10/16
P1-14	Guard watch-mode verification	**Low–Med**	1/2/4
P1-15	Restrict project roots	**Medium**	1/3/5
P1-16	Cancel-safe remote enable	**Low–Med**	1/2/4
P1-17	macOS/Linux validation	**Medium**	6/12/20
P2-01…08	DEMO stage, refactor, intent UX, cancel text, pairing/push, bundle size, CI Node runtime, UI E2E	**Low–Med**	67/129/215
P3-01…04	Agensi skills, Manager MCP wait tool, OpenCode adapter, configurable repairs	**Opt.**	41/77/132

12. Agensi commercialization assessment

Sources: Agensi's own pages, How to Sell Skills on Agensi and Agent Skills Marketplace, plus the SKILL.md specification guide. The Agensi connector was added during the audit but stayed in a "pending" state, so marketplace data could not be queried directly. Agensi's creator IP and ownership terms were **not found** and are marked UNKNOWN.

What Agensi accepts (per its documentation)

- A zip containing at least a `SKILL.md` with frontmatter `name` (≤64 chars, lowercase/hyphens) and `description` (≤1024 chars). Optional `scripts/`, `references/`, `assets/`.
- Automated 8-point scan: file structure, file types, dangerous command patterns, secrets, environment-variable harvesting, network access, obfuscation, prompt injection. Then manual review, typically 24–48 h.
- Skills are delivered "through the agent connection rather than downloadable files". Free or paid (one-time or subscription); creators keep 70%; Stripe Connect required. Testing on at least two agents (e.g. Claude Code and Codex CLI) is recommended.

Fit of Hydra

Question	Answer
Does full Hydra fit the skill format?	**No.** It is a 34.9k-line Electron app with native modules (node-pty), a daemon and a Firebase backend. Agensi distributes instruction content, not installers. Hydra Core would need its own distribution channel.
Separate installer needed?	Yes — signed platform installers (P0-05, P1-12).
Extractable standalone workflows?	Yes, as *method*, not code: the cross-agent protocol (`AI_TEAM.md`), evidence-based verification (`verification.ts` + `semanticResults.ts`), and the independent analysis/review prompt structure (`TaskCoordinator.ts:173, 229`). Today these are one-line prompts or internal docs; there are **0 SKILL.md files**.
External services	Hydra Core requires Claude Code and Codex CLI installed and authenticated by the buyer (their own Anthropic/OpenAI plans or API keys). Remote Control requires a Firebase project on the Blaze (pay-as-you-go) plan, because Cloud Functions need it.
Firebase operating cost	If you host the backend for customers, you pay for it. Current sync code writes about 28.8k task docs per hour per active session at 30 tasks (P1-08) — fix this before hosting.
Licensing	MIT allows resale with the licence notice retained. However, the upstream is free and public, so selling Hydra Core competes with the free original. Honest attribution is mandatory for credibility. Not legal advice.
Public GitHub repo	Anyone can build Hydra Core for free, so paid value must come from separate artefacts (skills, signed builds, support). Keep paid SKILL.md content out of the public repo.

Candidate products

Product	Exists today	Must be built	Standalone?	Dependencies	Install	Readiness
1. Hydra Core	Inherited app + working task pipeline	All P0 + most P1 (~330 h realistic)	Yes, as an app	Claude Code, Codex CLI, optional Firebase	High	**Not ready; not an Agensi item**
2. Planning skill	One-line analysis prompt (`:173`)	Full SKILL.md: decomposition, risks, acceptance criteria, JSON plan	Yes	None	Low	**8–14 h; low differentiation**

3. Code review skill	Review prompt + verdict schema (`:229`)	Checklist, severity rubric, structured verdict, optional "ask the other agent" instructions	Yes	Optionally a second CLI	Low	**10–16 h; crowded category**
4. Auto-fix skill	One-line repair prompt (`:246`)	Everything	Yes	None	Low	**Weak value alone; bundle with review**
5. Verification skill **(strongest)**	Evidence rules (`verification.ts`, `semanticResults.ts`)	SKILL.md "never claim tests passed without an observed exit code", plus a script that detects check commands across ecosystems and reports JSON evidence	Yes	Local toolchain	Low–Med (scripts will be scanned)	**12–24 h; differentiated**
6. Cross-agent collaboration skill **(discovered)**	`AI_TEAM.md` protocol: roles, consultation budget, edit ownership, disagreement escalation	Convert to SKILL.md with invocation recipes for `codex exec` / `claude -p` in read-only mode	Yes	Both CLIs	Low	**6–10 h; most ready**

No market demand, revenue or customer-interest figures are claimed here; none were measured.

13. Development roadmap

Phase	Scope (backlog IDs)	Opt. h	Real. h	Pess. h
0 · Honesty and hygiene (week 1)	P0-01, P0-04, P0-07, P0-09, P0-10	8	15	27
1 · Agensi skills (fast commercial path)	P3-01: collaboration, verification, review/plan skills; test on Claude Code and Codex CLI	25	45	80
2 · Core completion	P1-01, 02, 03, 04, 05, 11, 13, 14, 15, P2-04	52	98	164
3 · Remote control done right	P0-02, 03, 08, P1-06, 07, 08, 09, 10, 16	29	60	104
4 · Security, dependencies, packaging	P0-05, 06, P1-12, 17, P2-07, 08	33	68	123
5 · Docs, refactor, launch prep	P2-02, 03, 06 + docs/site	24	48	72
6 · Beta	Fixes from 5–10 external testers (plus 2–4 calendar weeks)	20	40	80
Hydra Core total (excluding Phase 1)		**~165**	**~330**	**~570**
Optional DEMO stage (P2-01)		20	40	70

Critical path (Hydra Core): provenance decision → own Firebase + config embedding → dependency/Electron upgrade with green CI → worktree isolation + final approval → multi-ecosystem verification → signing + first release → truthful docs → beta.

Quick wins (<1 day): lint fix, attribution, README truthfulness, `.env` and npx fixes, Firestore change-only sync, remote acknowledgement, cancel text. **Substantial work:** worktree-based approval flow, multi-ecosystem verification, remote permission safety, Electron upgrade, signing, cross-platform validation.

14. Final technical verdict

1. Is Hydra actually functional today?

Partly. The desktop build compiles and tests pass, and the coordinated task pipeline works on this machine (verified live). Remote Control does not work in released builds, and there is no released build.

2. Can it complete a real development task autonomously?

Yes, for a small npm-project fix with both Claude and Codex (T17: completed, independently re-verified). Not for Python/Rust/Go with shell verification; Claude-only implementation failed in testing; larger tasks are unproven.

3. Is its multi-agent architecture reliable?

The task core is well designed (bounded, evidence-gated, cancellable). Overall reliability is not established: CI is red, validation is Windows-only, there are swallowed errors, persistence scaling issues, and four overlapping orchestration paths.

4. Does remote mobile control genuinely work?

No, not for a real user. It cannot be configured in packaged builds; the live instance relies on the upstream author's Firebase project and an outdated protocol; it is prompt-only, with no task control or approvals.

5. What parts are unfinished or broken?

Remote Control, releases and distribution, non-npm verification, Claude-only implementation, DEMO and final APPROVAL stages, cost controls, signing, docs, and CI.

6. What are its most dangerous weaknesses?

(a) Presenting a largely inherited codebase as original work. (b) Live remote traffic flowing through a third party's backend. (c) Critical dependency advisories and an unsupported Electron. (d) Unattended agents writing directly into users' repositories with no final gate. (e) Remote keystrokes that may confirm agent permission dialogs.

7. How much engineering work remains?

About 330 h realistic (165–570 h range) for a commercially distributable Hydra Core; about 60 h (35–105) for honest hygiene plus a first set of Agensi skills.

8. Is it safe for another developer to install?

Not yet. Local-only use has reasonable guardrails for a tool that runs AI agents, but the dependency advisories, EOL Electron, unsigned binaries, cwd `.env` propagation, unpinned npx execution and direct repository writes must be fixed first. Do not ship Remote Control until P0-02/03/08 are done.

9. Is it ready to be sold?

No.

10. What exactly should I build or fix next?

In order: P0-01 attribution → P0-04 green CI → P0-07 truthful README → P0-09/P0-10 → publish one free skill (cross-agent collaboration) and one paid skill (evidence-based verification) on Agensi → then the Hydra Core critical path (P0-02/03 → P0-06 → P1-03 → P1-01 → P1-02 → P0-05/P1-12).

11. What should I absolutely NOT advertise yet?

"Remote/mobile control", "YOLO mode", "install via Homebrew/Releases", "signed builds", "works with any project/language", "autonomous end-to-end with demo and approval", "secure for production", "cost-controlled", "cross-platform tested", and any wording implying sole authorship of the Hydra desktop app.

12. Would I approve the commercial launch today?

No. I would approve publishing a *free, accurately attributed* open-source build of the task pipeline after Phase 0, and listing 1–2 well-tested SKILL.md skills on Agensi after Phase 1. I would not approve selling Hydra Core until the P0 list is closed and a beta has run.

15. Evidence appendix

Commands executed

Command	Result
`git clone …/Hydra-Agent-Orchestrator C:\hydra-audit\src`	OK (the scratch path was too long for git objects)
`git clone https://github.com/jpdlr/hydra upstream`; `git diff --shortstat d8ad561 fork`	113 files, +4,439/−804
`gh repo view` / `gh release list` / `gh run list` / `gh run view 38086741427 --log-failed`	Public, not marked fork; 0 releases; CI lint failures
`gh api repos/jpdlr/hydra`; `gh api repos/MoallaMelek/homebrew-hydra`	Upstream exists (MIT, 2026-02-07); tap 404
`npm ci`; `npm run typecheck`; `npm run lint`; `npx vitest run`; `npm run build`	OK; exit 0; exit 1 (6 errors); 236/236; exit 0
`npm audit [--omit=dev]` (root, hydra-remote, functions)	See T09
`hydra-remote: npm ci && npm run build`; `functions: npm ci && npm run build`	exit 0; exit 0
Sentinel `npx electron-vite build` with `MAIN_VITE_*` markers	0 occurrences in `out/`
`claude --help`; `codex exec --help`	Required flags present
`curl https://hydra-remote-moalla-260912.web.app/` and its JS bundle	200; authDomain `hydra-za.firebaseapp.com`; protocol includes `get_history`, `stop`, `restart`
`node audit-scripts/probes.mjs`	See T20–T22
`node audit-scripts/live-e2e.mjs … (5 scenarios)`	See T15–T19
`codex exec -s read-only` challenge review	F1–F3 partial, F4 refuted (downgraded here), F5–F10 confirmed; 5 new findings, 3 adopted as P1-04/05/06/16 after re-checking

Key file references

Area	References
Task pipeline	`electron/tasks/TaskCoordinator.ts:17-18, 26-52, 141-154, 156-278, 280-289, 291-319, 334-338`
Provider confinement	`electron/tasks/providerAdapters.ts:19-63`; `electron/agents/providers.ts:117-157`; `electron/headless/HeadlessOrchestrator.ts:94-121`; `electron/agents/cliExecution.ts:7-79`
Verification and evidence	`electron/tasks/verification.ts:6-34`; `electron/tasks/semanticResults.ts:8-52`; `electron/tasks/projectContext.ts:7-64`
Remote	`electron/remote/RemoteControlService.ts:37-57, 184-236, 400-462, 598-605`; `electron/remote/firebaseConfig.ts:11-71`; `electron/remote/mobileLink.ts:1-23`; `hydra-remote/src/hooks/useRemoteSession.ts:79-283`
Firebase	`firebase-backend/firestore.rules:1-73`; `firebase-backend/functions/src/index.ts:19-102`; `firebase-backend/.firebaserc:3`
Interactive agents	`electron/agents/AgentManager.ts:464-596, 801-838, 916-921, 967-1050`
Security boundaries	`electron/main.ts:68-73`; `electron/security/pathPolicy.ts:15-66`; `electron/daemon/DaemonServer.ts:107-114, 171-172, 455`; `electron/util/load-env.ts:38-47`; `electron/ipc/handlers.ts:642-700`

Packaging and CI	`electron-builder.yml:5-30`; `.github/workflows/release.yml:20-29, 54-73`; `.github/workflows/ci.yml`
Internal records	`.ai/decisions.md`; `.ai/handoffs/final.md`; `docs/debug/HYDRA-DEEP-AUDIT.md`; `docs/debug/HYDRA-CURRENT-STATE.md`

Secrets: no secret values are reproduced in this report. Firebase web API keys observed in the deployed bundle were intentionally not recorded.